Uppsala University

Parallel and Distributed Programming

1TD070

Assignment 2

Authors:
Anton von
Nandelstadh

April 16, 2026

1 Parallelisation

Algorithm 1 Parallel Stencil Algorithm

```
Root node scatters data to each node
procedure StencilApplication(processors)
    Exchange information about edges of local buffers
    for all processors do
        Apply stencil on local buffer
    end for
    Recursively apply StencilApplication
end procedure
Gather the data back to root node
```

We apply algorithm 1. To distribute the data, we first broadcast data about buffer sizes, and then scatter the data from the root node.

Listing 1: Broadcasting and scattering.

```
    MPI_Bcast(&global_num_values, 1, MPI_INT, 0,
       MPI_COMM_WORLD);
    num_values = global_num_values / p;

    input = malloc(sizeof(double) * num_values);

    MPI_Scatter(global_input, num_values, MPI_DOUBLE,
       input, num_values, MPI_DOUBLE, 0, MPI_COMM_WORLD)
       ;
```

In the beginning of each recursive for loop, we exchange information about the edges between the buffers. We implement this as persistent buffers, since we are exchanging information of the same format every time, and so using persistent buffers should greatly reduce overhead.

Listing 2: Exchanging information about buffer edges.

```
    for (int s = 0; s < num_steps; s++) {
        memcpy(high_s_buf, input + num_values - EXTENT,
           EXTENT * sizeof(double));
        memcpy(low_s_buf, input, EXTENT * sizeof(double)
           );
```

```
        MPI_Start(&recv_higher);
        MPI_Start(&recv_lower);
        MPI_Start(&send_higher);
        MPI_Start(&send_lower);

        MPI_Wait(&send_higher, &stat);
        MPI_Wait(&send_lower, &stat);
        MPI_Wait(&recv_higher, &stat);
        MPI_Wait(&recv_lower, &stat);
```

Finally, when we have computed all of our stencil applications, we need to gather the information from all of the nodes to the root, so that the root can write it into a file. Moreover, we should reduce our time measurements so the we only record the maximum.

Listing 3: Reducing and gathering.

```
    MPI_Reduce(&my_execution_time, &
       global_execution_time, 1, MPI_DOUBLE, MPI_MAX, 0,
        MPI_COMM_WORLD);
    MPI_Gather(output, num_values, MPI_DOUBLE,
       global_output, num_values, MPI_DOUBLE, 0,
       MPI_COMM_WORLD);
```

The sequential algorithm is a slightly simplified version of the original, where we collapse the three separate for loops into one.

Listing 4: Sequential algorithm.

```
        for (int i = 0; i < num_values; i++) {
            double result = 0;
            for (int j = 0; j < STENCIL_WIDTH; j++) {
                int id = i - EXTENT + j;
                double temp;
                if (id < 0) {
                    temp = low_r_buf[id + EXTENT];
                } else if (id >= num_values) {
                    temp = high_r_buf[id - num_values];
                } else {
                    temp = input[id];
                }
                result += STENCIL[j] * temp;
            }
```

```
            output[i] = result;
        }
```

2 Results

We ran our performance experiments on Pelle, using varying numbers of nodes. We wanted to measure both strong and weak scaling, so we did two sets of experiments. First, we kept the number of inputs constant while varying the number of cores to measure strong scaling. This can be seen in figure 2 and table 2. Second, we wanted to measure weak scaling, so we kept the input per core constant. That is, we increased input size as we increased the number of cores. This can be seen in figure 1 and table 1.

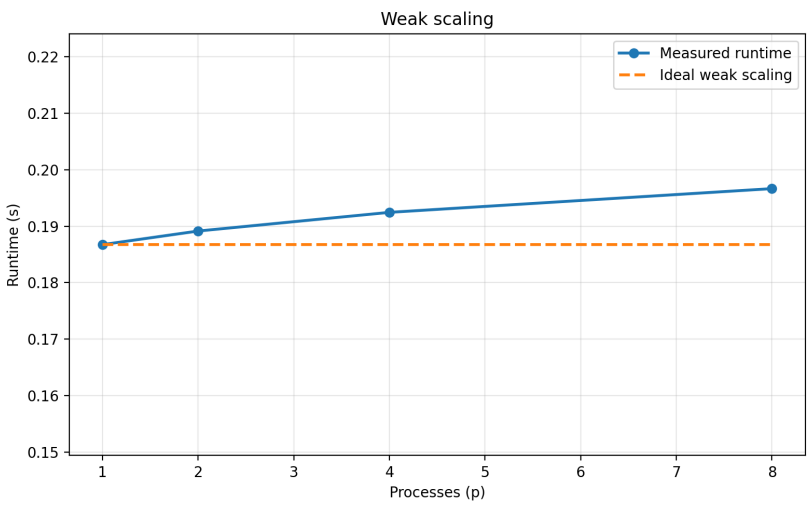

Figure 1: Weak scaling

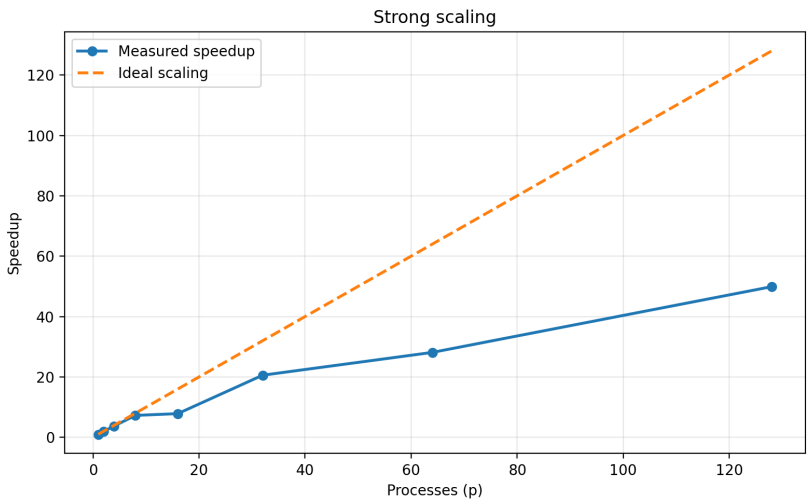

Figure 2: Strong scaling

n	p	time
1000000	1	0.186745
2000000	2	0.189145
4000000	4	0.192455
8000000	8	0.196656

Table 1: Table for weak scaling.

n	p	time
8000000	1	1.497752
8000000	2	0.752445
8000000	4	0.399575
8000000	8	0.206039
8000000	16	0.190692
8000000	32	0.072775
8000000	64	0.053237
8000000	128	0.030011

Table 2: Table for strong scaling.

3 Discussion

The scaling we saw was almost linear up to 8 nodes, and then started dropping off. The drop-off was particularly noticeable after 32 nodes, which might be because of the CPUs on Pelle having 48 cores, and going beyond that meaning slower communication between nodes, thus increasing overhead. However, some of this drop-off is likely also explained simply by Amdahl's/Gustafson's law. An interesting extension would be to see if the drop-off would be as large for larger input sizes.

Because of restrictions on the input data, we were only able to measure weak scaling up to 8 nodes. We would have needed larger datasets to measure on more nodes, and an extension would be to create synthetic data to enable us to do this. However, with the current data, the weak scaling does not tell us very much, since the strong scaling was already almost linear from 1 to 8 nodes.